

QairtBackend

Updated: Jul 02, 2026

80-63442-10

Rev: AL

Provide Feedback

Note

Some methods in this module are not yet implemented in the current release and will raise an exception if called. See the C API for full functionality.

Include: `#include "QairtCppApi/QairtBackend.hpp"`

C++ wrapper for the QAIRT Backend API.

namespace qairt

Enums

enum class BackendError : std::underlying_type_t<QairtBackend_Error_t>

Error codes returned by QAIRT backend operations.

Enumerator	Description
NoError	Operation succeeded.
MemAlloc	Memory allocation failure.
UnsupportedPlatform	Backend creation attempted on an unsupported platform.



CannotInitialize	Backend failed to initialize.
TerminateFailed	Failed to free allocated resources during termination.
NotSupported	Requested functionality is not supported by this backend.
InvalidArgument	An argument to the operation was invalid.
OpPackageNotFound	The specified op package library could not be found.
OpPackageIfProviderNotFound	The interface provider symbol was not found in the op package.
OpPackageRegistrationFailed	Op package registration failed.
OpPackageUnsupportedVersion	The op package interface version is not supported.
OpPackageDuplicate	An op with the same package and op name is already registered.
InconsistentConfig	Backend configuration is inconsistent across create calls.
InvalidHandle	The provided backend handle is not valid.
InvalidConfig	One or more configuration values are invalid.
Undefined	An undefined or unknown error occurred.

Provide Feedback

Values:

enumerator NoError = QAIRT_BACKEND_NO_ERROR

enumerator MemAlloc = QAIRT_BACKEND_ERROR_MEM_ALLOC

**enumerator UnsupportedPlatform =
QAIRT_BACKEND_ERROR_UNSUPPORTED_PLATFORM**



enumerator CannotInitialize = QAIRT_BACKEND_ERROR_CANNOT_INITIALIZE

enumerator TerminateFailed = QAIRT_BACKEND_ERROR_TERMINATE_FAILED

enumerator NotSupported = QAIRT_BACKEND_ERROR_NOT_SUPPORTED

enumerator InvalidArgument = QAIRT_BACKEND_ERROR_INVALID_ARGUMENT

**enumerator OpPackageNotFound =
QAIRT_BACKEND_ERROR_OP_PACKAGE_NOT_FOUND**

**enumerator OpPackageIfProviderNotFound =
QAIRT_BACKEND_ERROR_OP_PACKAGE_IF_PROVIDER_NOT_FOUND**

**enumerator OpPackageRegistrationFailed =
QAIRT_BACKEND_ERROR_OP_PACKAGE_REGISTRATION_FAILED**

**enumerator OpPackageUnsupportedVersion =
QAIRT_BACKEND_ERROR_OP_PACKAGE_UNSUPPORTED_VERSION**

**enumerator OpPackageDuplicate =
QAIRT_BACKEND_ERROR_OP_PACKAGE_DUPLICATE**

enumerator InconsistentConfig = QAIRT_BACKEND_ERROR_INCONSISTENT_CONFIG

enumerator InvalidHandle = QAIRT_BACKEND_ERROR_INVALID_HANDLE

enumerator InvalidConfig = QAIRT_BACKEND_ERROR_INVALID_CONFIG

enumerator Undefined = QAIRT_BACKEND_ERROR_UNDEFINED

```
enum class ErrorReportingConfigLevel :
std::underlying_type_t<QairtErrorReporting_Config_Level_t>
```

Verbosity levels for the error reporting configuration.

Enumerator	Description
Brief	Collect basic summary information about each error.
Detailed	Collect detailed, memory-resident error information.
Undefined	Level is unset or unknown.

Values:

```
enumerator Brief = QAIRT_ERROR_REPORTING_LEVEL_BRIEF
```

```
enumerator Detailed = QAIRT_ERROR_REPORTING_LEVEL_DETAILED
```

```
enumerator Undefined = QAIRT_ERROR_REPORTING_LEVEL_UNDEFINED
```

```
class Backend : public qairt::ApiType<Backend, QairtBackend_V1_t>
```

```
#include <QairtBackend.hpp>
```

Wrapper for a QAIRT Backend handle.

Obtained via Api::createBackend().

Public Functions

```
Backend() noexcept = default
```

```
Backend(const Backend&) = delete
```

```
Backend(Backend&&) noexcept = default
```

```
Backend &operator=(const Backend&) = delete
```

```
Backend &operator=(Backend&&) noexcept = default
```

```
inline Backend(const std::shared_ptr<ApiTable> &apiTable, QairtBackend_Handle_t  
handle)
```

```
inline void setConfig(const BackendConfiguration &config)
```

Set configuration options on this backend after creation.

See also

[QairtBackend_setConfig](#)

Parameters

config – [in] The backend configuration to apply.

Throws

[qairt::Exception](#) – on invalid handle, invalid config, or unsupported feature.

```
inline void registerOpPackage(const char *packagePath, const char  
*interfaceProvider, const char *target)
```

Register an op package library with this backend.

Loads the shared library at *packagePath* and registers its operations using the interface provider function *interfaceProvider*. An optional *target* platform string restricts registration to a specific processing unit.

QairtBackend_registerOpPackage

Parameters

- **packagePath** – [in] Path on disk to the op package shared library. Must not be NULL.
- **interfaceProvider** – [in] Name of the interface provider function exported by the op package library. Must not be NULL.
- **target** – [in] Optional target platform string. NULL applies no target restriction.

Throws

`qairt::Exception` – on:

- invalid handle
- NULL *packagePath* or *interfaceProvider*
- library not found
- interface provider symbol not found
- registration failure
- unsupported op package interface version
- duplicate op registration

```
inline void registerOpPackage(const std::string &packagePath, const std::string  
&interfaceProvider, const std::string &target)
```

Wrapper which allows for `std::string` path arguments instead of `const char*`.

See also

[Backend::registerOpPackage\(const char*, const char*, const char*\)](#)

```
inline std::vector<BackendOperationName> getSupportedOperations() const
```

Get all operations supported by this backend, including built-in ops.



QairtBackend_getSupportedOperations

Throws

`qairt::Exception` – on invalid handle.

Returns

Vector of `BackendOperationName` descriptors, one per supported operation.

```
inline void validateOpConfig(const OpConfig &opConfig)
```

Validate an op configuration against the appropriate registered op package.

The backend selects the op package for validation based on attributes of *opConfig*.

See also

[QairtBackend_validateOpConfig](#)

Parameters

`opConfig` – [in] The op configuration to validate.

Throws

`qairt::Exception` – on:

- invalid handle
- validation failure
- validation not supported by this backend
- no matching op package found

```
inline void validateContextBinary(ApiTypeRef<const Device&> device,  
ApiTypeRef<const ContextBinaryBuffer&> contextBinary, ApiTypeRef<const  
ContextConfiguration&> contextConfig)
```

Validate a context binary against a device and context configuration.

is loaded via `createContextFromBinary()`.

Parameters

- **device** – [in] The device on which the binary would be loaded.
- **contextBinary** – [in] The context binary buffer to validate.
- **contextConfig** – [in] The context configuration to validate against.

Throws

`qairt::Exception` – on invalid handle or validation failure.

```
inline Context createContext(ApiTypeRef<const ContextConfiguration>  
contextConfig = {})
```

Create a context using this backend.

Creates a context with no device (uses backend default) and an optional context configuration.

Parameters

contextConfig – [in] `Context` configuration. Optional.

Throws

`qairt::Exception` – on invalid handle or configuration error.

Returns

A new `Context`.

```
inline Context createContext(ApiTypeRef<const Device> device, ApiTypeRef<const  
ContextConfiguration> contextConfig)
```

Create a context for a specific device using this backend.

Parameters

- **device** – [in] The device on which to create the context.
- **contextConfig** – [in] `Context` configuration. Optional.

`qairt::Exception` – on invalid handle or configuration error.

Returns

A new `Context`.

```
inline Context createContextFromBinary(ApiTypeRef<const Device&> device,  
ApiTypeRef<const ContextConfiguration&> contextConfig, ApiTypeRef<const  
ContextBinaryBuffer&> contextBinaryBuffer, ApiTypeRef<const Signal&> signal = {},  
ApiTypeRef<const Profile&> profile = {})
```

Create a context from a serialized context binary.

Pass a `Signal` to enable aborting or timing out the load operation.

Parameters

- `device` – [in] The device on which to load the context binary.
- `contextConfig` – [in] `Context` configuration. Optional.
- `contextBinaryBuffer` – [in] The serialized context binary to load.
- `signal` – [in] Optional signal to control the load operation.
- `profile` – [in] Optional profile handle to collect load-time events.

Throws

`qairt::Exception` – on invalid handle, binary incompatibility, or configuration error.

Returns

A new `Context`.

```
inline Profile createProfile(uint32_t level)
```

Create a profiling handle at the specified granularity.

See also

`QairtBackend_createProfile`

level – [in] Granularity level at which events should be collected.

Throws

`qairt::Exception` – on invalid handle, unsupported profiling level, or memory error.

Returns

A new [Profile](#).

```
inline std::shared_ptr<Profile> createSharedProfile(uint32_t level)
```

Create a shared profiling handle at the specified granularity.

See also

[QairtBackend_createProfile](#)

Parameters

level – [in] Granularity level at which events should be collected.

Throws

`qairt::Exception` – on invalid handle, unsupported profiling level, or memory error.

Returns

A `std::shared_ptr` to a new [Profile](#).

```
inline Signal createSignal(ApiTypeRef<const SignalConfiguration> signalConfig = {})
```

Create a new signal object associated with this backend.

Signals are used to control the execution of API calls that accept them (e.g., `Graph::execute`, `Context::createFromBinary`). The created signal is idle and immediately available for use.

The signal is backend-scoped: the backend allocates any required synchronization primitives and validates signal support. A signal may only be used with the backend that created it.

signalConfig – [in] Configuration for the signal. Optional.

Throws

`qairt::Exception` – on:

- invalid handle
- signals not supported by this backend
- invalid signal configuration
- memory allocation failure

Returns

A new `Signal` object.

Provide Feedback

Private Functions

```
inline Backend(const std::shared_ptr<ApiTable> &apiTable, ApiTypeRef<const  
Log&> log, ApiTypeRef<const BackendConfiguration&> config = {})
```

Private Members

```
detail::crossable<detail::set_only<BackendConfiguration>, nullptr,  
&interface_type::setConfig> m_memory
```

Staging storage for a `BackendConfiguration` being crossed to the C layer.

Friends

```
friend class Api
```

```
class BackendConfiguration : public qairt::ApiType<BackendConfiguration,  
QairtBackend_ConfigV1_t>
```

```
#include <QairtBackend.hpp>
```

Configuration object for backend creation and reconfiguration.



options before passing to `Api::createBackend()` or `Backend::setConfig()`. All setter methods return `*this` to support method chaining.

Public Functions

BackendConfiguration() noexcept = default

BackendConfiguration(const BackendConfiguration&) = delete

BackendConfiguration(BackendConfiguration&&) noexcept = default

BackendConfiguration &operator=(const BackendConfiguration&) = delete

BackendConfiguration &operator=(BackendConfiguration&&) noexcept = default

inline BackendConfiguration &setCustomConfig(const BackendCustomConfig &backendCustomConfig)

Set a single backend-specific custom configuration item on this configuration.

See also

[QairtBackend_Config_setCustomConfigs](#)

Parameters

backendCustomConfig – [in] The custom configuration item whose handle will be applied.

Throws

`qairt::Exception` – on invalid handle.

Returns

```
inline BackendConfiguration &setCustomConfigs(const  
BackendCustomConfiguration &config)
```

Set a collection of backend-specific custom configuration items on this configuration.

See also

[QairtBackend_Config_setCustomConfigs](#)

Parameters

config – [in] The custom configuration collection whose handles will be applied.

Throws

[qairt::Exception](#) – on invalid handle.

Returns

Reference to this object for method chaining.

```
inline uint32_t getNumPlatformOptions() const
```

Get the number of platform options set on this configuration.

See also

[QairtBackend_Config_getNumPlatformOptions](#)

Throws

[qairt::Exception](#) – on invalid handle.

Returns

Number of platform option strings currently set.

```
inline std::string_view getPlatformOptionAt(uint32_t idx) const
```

Get the platform option string at the specified index.



QairtBackend_Config_getPlatformOptionAt

Parameters

idx – [in] Zero-based index into the platform options list. Must be less than [getNumPlatformOptions\(\)](#).

Throws

[qairt::Exception](#) – on invalid handle or out-of-range index.

Returns

The null-terminated platform option key-value pair string at *idx*, or an empty view if the stored pointer is null.

Provide Feedback

```
inline std::vector<std::string_view> getPlatformOptions() const
```

Get all platform option strings set on this configuration.

See also

[QairtBackend_Config_getNumPlatformOptions](#)

Throws

[qairt::Exception](#) – on invalid handle.

Returns

Vector of platform option key-value pair strings.

```
inline BackendConfiguration &setPlatformOptions(const std::vector<const char*>
&platformOptions)
```

Set the platform options on this configuration from an array of C strings.

See also

[QairtBackend_Config_setPlatformOptions](#)

Parameters

platformOptions – [in] Array of null-terminated platform option strings.



`qairt::Exception` – on invalid handle or invalid options.

Returns

Reference to this object for method chaining.

inline `BackendConfiguration` &setPlatformOptions(const std::vector<std::string> &platformOptions)

Wrapper which allows for `std::string` platform option values instead of `const char*`.

See also

[BackendConfiguration::setPlatformOptions\(const std::vector<const char*>&\)](#)

inline `BackendConfiguration` &setPlatformOptions(const std::vector<std::string_view> &platformOptions)

Wrapper which allows for `std::string_view` platform option values instead of `const char*`.

See also

[BackendConfiguration::setPlatformOptions\(const std::vector<const char*>&\)](#)

inline `BackendConfiguration` &resetPlatformOptions()

Clear all platform options from this configuration.

See also

[QairtBackend_Config_setPlatformOptions](#)

Throws

`qairt::Exception` – on invalid handle.

Returns

```
inline std::optional<std::reference_wrapper<ErrorReportingConfig>>  
getErrorReportingConfig()
```

Get the error reporting configuration attached to this backend configuration.

See also

[QairtBackend_Config_getErrorReportingConfig](#)

Throws

`qairt::Exception` – on invalid handle.

Returns

A reference wrapper to the attached [ErrorReportingConfig](#), or an empty optional if none has been set.

```
inline std::optional<std::reference_wrapper<ErrorReportingConfig>>  
getErrorReportingConfig() const
```

Get the error reporting configuration attached to this backend configuration.

See also

[QairtBackend_Config_getErrorReportingConfig](#)

Throws

`qairt::Exception` – on invalid handle.

Returns

A const reference wrapper to the attached [ErrorReportingConfig](#), or an empty optional if none has been set.

```
inline void setErrorReportingConfig(const ErrorReportingConfig  
&errorReportingConfig)
```

Attach an error reporting configuration to this backend configuration.

QairtBackend_Config_setErrorReportingConfig

Parameters

errorReportingConfig – [in] The error reporting configuration to attach.

Throws

`qairt::Exception` – on invalid handle.

Private Functions

```
inline void prepareToCross() const
```

```
inline void updateAfterCross() const
```

```
inline explicit BackendConfiguration(const std::shared_ptr<ApiTable> &apiTable)
```

Private Members

```
std::optional<detail::crossable<detail::non_owning<ErrorReportingConfig>,  
&interface_type::getErrorReportingConfig, &interface_type::setErrorReportingConfig>>  
m_errorReportingConfig
```

Optional error reporting configuration cross-linked to the C handle.

Friends

```
friend class Api
```

```
class BackendCustomConfig : public qairt::CustomConfigType
```

```
#include <QairtBackend.hpp>
```

Abstract base class for a single backend-specific custom configuration item.



[BackendConfiguration::setCustomConfig\(\)](#). Refer to the backend documentation for the concrete subclass and valid handle values.

Public Functions

```
virtual ~BackendCustomConfig() = default
```

```
virtual QairtBackend\_CustomConfigHandle\_t getCustomConfigHandle() const = 0
```

Get the underlying C handle for this custom configuration item.

Returns

The backend-specific custom configuration handle.

Provide Feedback

Protected Functions

```
BackendCustomConfig() = default
```

```
BackendCustomConfig(const BackendCustomConfig&) = default
```

```
BackendCustomConfig(BackendCustomConfig&&) noexcept = default
```

```
BackendCustomConfig &operator=(const BackendCustomConfig&) = default
```

```
BackendCustomConfig &operator=(BackendCustomConfig&&) noexcept = default
```

```
class BackendCustomConfiguration
```



Abstract base class for a collection of backend-specific custom configuration items.

Subclass this to provide multiple backend-specific configuration handles to [BackendConfiguration::setCustomConfigs\(\)](#). Refer to the backend documentation for the concrete subclass and valid handle values.

Public Functions

```
virtual ~BackendCustomConfiguration() = default
```

```
virtual std::vector<QairtBackend\_CustomConfigHandle\_t> getCustomConfigs()  
const = 0
```

Get the list of underlying C handles for this custom configuration collection.

Returns

Vector of backend-specific custom configuration handles.

Protected Functions

```
BackendCustomConfiguration() = default
```

```
BackendCustomConfiguration(const BackendCustomConfiguration&) = default
```

```
BackendCustomConfiguration(BackendCustomConfiguration&&) noexcept = default
```

```
BackendCustomConfiguration &operator=(const BackendCustomConfiguration&) =  
default
```

```
BackendCustomConfiguration &operator=(BackendCustomConfiguration&&)
noexcept = default
```

class BackendOperationName

```
#include <QairtBackend.hpp>
```

Name descriptor for a single operation supported by a backend.

Obtained from [Backend::getSupportedOperations\(\)](#). All string views are non-owning references to backend-managed memory.

Public Functions

```
constexpr BackendOperationName() noexcept = default
```

```
constexpr BackendOperationName(const BackendOperationName&) noexcept =
default
```

```
constexpr BackendOperationName(BackendOperationName&&) noexcept = default
```

```
constexpr BackendOperationName &operator=(const BackendOperationName&)
noexcept = default
```

```
constexpr BackendOperationName &operator=(BackendOperationName&&)
noexcept = default
```

```
inline constexpr BackendOperationName(std::string_view packageName,
std::string_view name, std::string_view target) noexcept
```

Parameters

- **packageName** – [in] Name of the op package that provides this operation.
- **name** – [in] Name of the operation within the op package.
- **target** – [in] Target platform for this operation entry. May be empty if the backend does not distinguish targets.

```
inline const std::string_view &getPackageName() const noexcept
```

Get the op package name for this operation.

See also

[QairtBackend_OperationName_getPackageName](#)

Returns

Name of the op package that provides this operation.

```
inline const std::string_view &getName() const noexcept
```

Get the operation name within its op package.

See also

[QairtBackend_OperationName_getName](#)

Returns

Name of the operation within its package.

```
inline const std::string_view &getTarget() const noexcept
```

Get the target platform for this operation entry.

See also

[QairtBackend_OperationName_getTarget](#)

Returns

Private Members

std::string_view m_packageName

The op package that provides this operation.

std::string_view m_name

The name of the operation within its package.

std::string_view m_target

The target platform for which this operation entry is registered.

**class ErrorReportingConfig : public [qairt::ApiType](#)<[ErrorReportingConfig](#),
[QairtErrorReporting_Config_V1_t](#)>**

#include <QairtBackend.hpp>

Configuration object for backend error reporting behavior.

Controls how much detail is captured when errors occur and how much memory is reserved for error data. Obtained via [Api::createBackend\(\)](#) or set on a [BackendConfiguration](#) before backend creation.

Public Functions

ErrorReportingConfig() = default

inline [ErrorReportingConfigLevel](#) getReportingLevel() const

Get the reporting verbosity level for this error reporting configuration.

See also

[QairtErrorReporting_Config_getReportingLevel](#)

Throws

Returns

The current reporting level.

```
inline void setReportingLevel(ErrorReportingConfigLevel level)
```

Set the reporting verbosity level for this error reporting configuration.

See also

[QairtErrorReporting_Config_setReportingLevel](#)

Parameters

level – [in] Desired reporting verbosity. Must be a valid enumerator.

Throws

[qairt::Exception](#) – on invalid handle or invalid level.

```
inline uint32_t getStorageLimit() const
```

Get the memory storage limit for this error reporting configuration.

See also

[QairtErrorReporting_Config_getStorageLimit](#)

Throws

[qairt::Exception](#) – on invalid handle.

Returns

Storage limit in kilobytes.

```
inline void setStorageLimit(uint32_t limit)
```

Set the memory storage limit for this error reporting configuration.

See also

Parameters

limit – [in] Maximum memory reserved for error information, in kilobytes.

Throws

`qairt::Exception` – on invalid handle or invalid limit.

Private Functions

```
inline ErrorReportingConfig(const std::shared_ptr<ApiTable> &apiTable,  
    ErrorReportingConfigLevel level, uint32_t storageLimit)
```

```
inline explicit ErrorReportingConfig(const std::shared_ptr<ApiTable> &apiTable)
```

Friends

```
friend class Api
```

```
friend class ::qairt::ApiType
```

Provide Feedback

[< Previous](#)

[Next >](#)

[QairtTensor](#)

[QairtDevice](#)

May contain U.S. and international export controlled information

Light Dark Auto



Qualcomm relentlessly innovates to deliver intelligent computing everywhere, helping the world tackle some of its most important challenges. Our leading-edge AI, high performance, low-power computing, and unrivaled connectivity deliver proven solutions that transform major industries. At Qualcomm, we are engineering human progress.



Provide Feedback

Quick links

- Products
- Support
- Partners
- Contact us
- Developer

Company info

- About us
- Careers
- Investors
- News & media
- Our businesses
- Email Subscriptions

Stay connected

Get the latest Qualcomm and industry information delivered to your inbox.



Manage your subscription



© Qualcomm Technologies, Inc. and/or its affiliated companies.

Snapdragon and Qualcomm branded products are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patented technologies are licensed by Qualcomm Incorporated.

Note: Certain services and materials may require you to accept additional terms and conditions before accessing or using those items.



units within the Qualcomm corporate structure, as applicable.

Qualcomm Incorporated includes our licensing business, QTL, and the vast majority of our patent portfolio. Qualcomm Technologies, Inc., a subsidiary of Qualcomm Incorporated, operates, along with its subsidiaries, substantially all of our engineering, research and development functions, and substantially all of our products and services businesses, including our QCT semiconductor business.

Materials that are as of a specific date, including but not limited to press releases, presentations, blog posts and webcasts, may have been superseded by subsequent events or disclosures.

Nothing in these materials is an offer to sell or license any of the services or materials referenced herein.